Daffa Khairul Ammar / 李凱 - F1151035

Software Engineering Midterm Project Report

1. Project Motivation and Problem Statement

This project addresses the task of determining the clockwise order of three labels, A, B, and C, placed around a closed curve in an image. A person can inspect the diagram and trace the order manually, but a prediction system can provide a consistent automated result for images with the same format. Such a system can support the automatic checking or categorization of labelled diagram images when the required result depends on spatial order.

The task is a supervised binary image-classification problem. Each input image belongs to one of two classes:

- + when the clockwise order is A → B → C
- - when the clockwise order is A → C → B

The input is one PNG image containing a coloured closed curve and the labels A, B, and C. The output is a predicted class, either + or -, together with a confidence score. During training, the project maps ABC to binary target 1 and ACB to binary target 0. This mapping defines the correct labels for the training data. It does not manually trace the curve during prediction. The CNN learns the visual relationship between the image pixels and the target class from labelled examples.

The goal is to build a complete prediction system rather than only train a model. The project provides a saved CNN checkpoint, a standalone inference.py script for predicting one image, and program.py as a command-line interface for choosing a saved model, reviewing its test metrics, testing an image, or training a new named model.

2. Dataset Description

The dataset was provided as labelled PNG images. It contains 200 images stored in the figures/ directory, contains one coloured closed curve with three labels: A, B, and C. The filename contains an identifier and the class annotation.

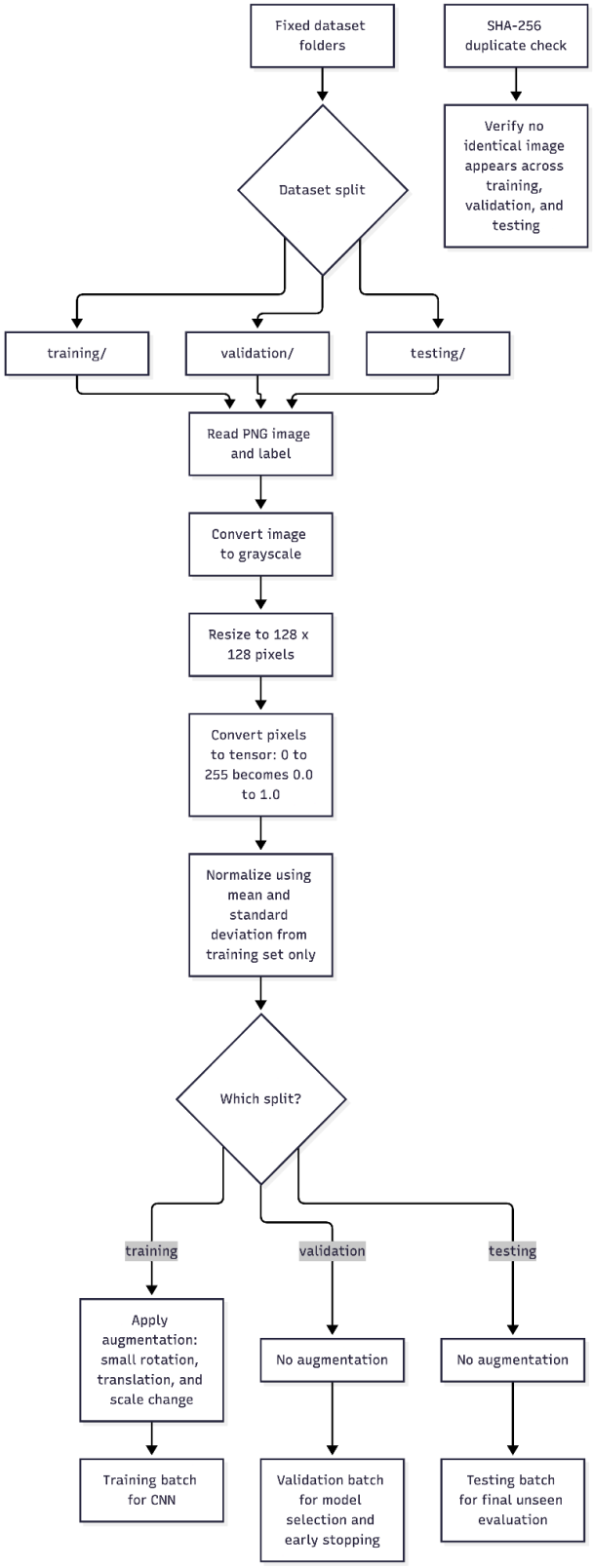
The label is determined by reading the labelled points clockwise around the closed curve, starting from A. If B appears after A and C appears after B, the image belongs to class +. If C appears after A and B appears after C, the image belongs to class -. This label rule is used to annotate the training examples. The CNN does not receive the sequence as text. It learns from the image pixels and their corresponding labels. The dataset contains 104 positive images and 96 negative images.

The dataset uses a fixed split based on image ID. The split is not random and not stratified. This decision keeps the experiment reproducible because every training run uses the same images in each split.

Split	Image IDs	Images	Positive (+)	Negative (-)	Ratio
Training	0-139	140	67	73	70%
Validation	140-169	30	18	12	15%
Testing	170-199	30	19	11	15%

The training split updates model weights. The validation split selects the checkpoint with the lowest validation loss and controls early stopping. The testing split is held out until final evaluation. The program calculates image hashes before training and stops if identical image content appears in more than one split. Normalization values are calculated from training images only, and test images are never used to fit preprocessing or choose model parameters.

3. Data Preprocessing Pipeline



The preprocessing pipeline prepares every image in a consistent format before it reaches the CNN. Each PNG is first converted to grayscale, resized to 128 × 128 pixels, converted into a tensor, and normalized.
Grayscale conversion removes colour as a main source of variation. The class depends on the clockwise order of A, B, and C, not on the colour of the curve. Resizing gives every input the same spatial size, which is required by the CNN. Converting the image to a tensor changes pixel values from the range 0 to 255 into the range 0 to 1. Normalization then centres and scales pixel values using the mean and standard deviation calculated from the training split.

Training images receive additional affine augmentation. The program applies small rotations, translations, and scaling changes so the CNN sees slightly different versions of the same visual structure. This can reduce overfitting to exact positions in the training images. Horizontal and vertical flips are excluded because a reflection can reverse the clockwise label order and change the correct class.

Validation, testing, and inference images do not receive random augmentation. They only use deterministic preprocessing: grayscale conversion, resizing, tensor conversion, and normalization. This makes validation and test results stable and comparable.

The main data-leakage risk is computing normalization values from all images before the train-validation-test split. The program avoids this problem by calculating the mean and standard deviation from the 140 training images only. Validation and test images use those saved training values but never contribute to them.

Training only normalization:

```
def calculate_training_normalization(
    samples: list[ImageSample],
) -> tuple[float, float]:
    pixel_sum = 0.0
    squared_pixel_sum = 0.0
    pixel_count = 0

    for sample in samples:
        with Image.open(sample.path) as image:
            pixels = np.asarray(
                image.convert("L").resize((IMAGE_SIZE, IMAGE_SIZE)),
                dtype=np.float32,
            ) / 255.0
        pixel_sum += float(pixels.sum())
        squared_pixel_sum += float((pixels * pixels).sum())
        pixel_count += pixels.size

    mean = pixel_sum / pixel_count
    variance = max(squared_pixel_sum / pixel_count - mean * mean, 1e-8)
```

```
    return mean, variance**0.5
```

Separate transforms for training and evaluation:

```
normalize = transforms.Normalize((mean,), (standard_deviation,))

training_transform = transforms.Compose(
    [
        transforms.RandomAffine(
            degrees=8,
            translate=(0.04, 0.04),
            scale=(0.95, 1.05),
            fill=255,
        ),
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        normalize,
    ]
)

evaluation_transform = transforms.Compose(
    [
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        normalize,
    ]
)

# training_transform includes random augmentation. evaluation_transform does not.
```

Applying the correct transform to each split:

```
mean, standard_deviation = calculate_training_normalization(
    training_samples
)
training_transform, evaluation_transform = create_transforms(
    mean,
    standard_deviation,
)

loaders = {
    "training": DataLoader(
        ClockwiseImageDataset(training_samples, training_transform),
        batch_size=args.batch_size,
        shuffle=True,
    ),
```

```
    "validation": DataLoader(
        ClockwiseImageDataset(validation_samples, evaluation_transform),
        batch_size=args.batch_size,
    ),
    "testing": DataLoader(
        ClockwiseImageDataset(testing_samples, evaluation_transform),
        batch_size=args.batch_size,
    ),
}
```

The relevant implementation is in train.py (line 124), while inference restores the same preprocessing values from the selected checkpoint in inference.py (line 32).

4. Model Architecture and Algorithm Choice

Two models were used in this project: a traditional baseline model and a convolutional neural network.

1. Baseline model: Logistic Regression
   The baseline model is Logistic Regression. Each grayscale image is resized to 64 × 64 pixels, flattened into one long vector, and then classified as + or -.

   Logistic Regression is included as a reference point. It shows how well a simple linear model can classify the images using raw pixel values without learning spatial features. However, flattening an image removes much of the relationship between nearby pixels. It cannot directly represent that one letter or curve segment is above, below, or beside another part of the figure.

2. Main model
   The main model's name is ClockwiseCNN (as straightforward as it gets), it receives images with shape [batch size, 1, 128, 128].

Stage	Layers	Purpose
Block 1	Conv2d 1→16, BatchNorm, ReLU, MaxPool	Conv2d 1→16, BatchNorm, ReLU, MaxPool
Block 2	Conv2d 16→32, BatchNorm, ReLU, MaxPool	Combines simple features into local patterns around labels and the curve.
Block 3	Conv2d 32→64, BatchNorm, ReLU	Learns higher-level spatial relationships across larger areas of

		the image.
Spatial pooling	AdaptiveAvgPool2d to 4x4	Preserves coarse positional information such as upper lower, left, and right regions.
Classifier	Flatten, Linear 1024-> 128, ReLU, Dropout 0.4, Linear 128->1	Combines spatial features and produces one binary-classification logit.

The 4x4 spatial grid is important for this task. The model must distinguish the relative order of A, B, and C around the figure. A global 1 × 1 average pool would discard too much positional information. The 4 × 4 grid keeps a simplified map of where important features appear before the final classifier makes its decision.

BatchNorm2d stabilizes intermediate feature values during training. ReLU adds non-linearity, allowing the network to learn more complex visual relationships than a purely linear model. MaxPool2d reduces image size while keeping strong activations, which lets later layers observe a larger part of the original image. Dropout(0.4) is used in the classifier to reduce overfitting by randomly disabling part of the hidden activations during training.

Loss function and optimizer
The CNN produces one logit per image. The target is 1 for class + and 0 for class -, so BCEWithLogitsLoss is appropriate for this binary classification task.

BCEWithLogitsLoss combines sigmoid activation and binary cross-entropy in one numerically stable function. The model does not need a separate sigmoid layer during training. A positive logit supports class +, while a negative logit supports class -.

The optimizer is AdamW with a learning rate of 0.001 and weight decay of 0.0001. AdamW adjusts each parameter using its gradient history, which is useful when different CNN weights learn at different speeds. Weight decay discourages excessively large weights and provides additional regularization.

The training configuration is:

Hyperparameter	Value	Reason
Input size	128x128	Preserves label and curve detail while keeping computation manageable.
Batch size	16	Appropriate for the small dataset and

		available device memory.
Maximum epochs	100	Gives the CNN enough opportunities to learn without fixing the final epoch in advance.
Learning rate	0.001	A standard starting value for AdamW.
Weight decay	0.0001	Adds regularization to reduce overfitting.
Dropout	0.4	Reduces dependence on individual classifier activations.
Early-stopping patience	20 epochs	Stops training when validation loss no longer improves.

Why these models were chosen
Logistic Regression provides a simple baseline. If the CNN performs better than Logistic Regression, the comparison supports the idea that learned spatial features are useful for this image task.

The CNN is suitable because it can learn local visual features and combine them into spatial relationships. It can learn patterns involving the letter positions, curve segments, and their coarse locations in the image.

The main advantage of the CNN is its ability to preserve spatial structure. Its main limitations are the small dataset size, the possibility of overfitting, and lower interpretability. The model can produce a confidence score, but it cannot explain its decision as clearly as a handwritten geometric rule.

For the implementation, see model.py (line 8) and the optimizer/loss configuration in train.py (line 271).

5. Experimental Design and Training Process

Experiment environment
The experiment was implemented in Python 3.11.15. The project uses PyTorch 2.15.0 and torchvision 0.29.2 for CNN training and image preprocessing. Scikit-learn 1.9.1 is used for the Logistic Regression baseline and classification metrics.

Training was performed on a MacBook Air with an Apple M5 chip and 16 GB memory. PyTorch detected Apple Metal Performance Shaders (MPS), so the CNN used the MPS backend instead of the CPU.

The random seed is set for Python, NumPy, and PyTorch. It makes model initialization, augmentation, and DataLoader shuffling reproducible. It does not change the fixed membership of the training, validation, and testing folders.

Training procedure
The experiment uses fixed dataset folders: 140 images for training, 30 images for validation, and 30 images for testing. Before training starts, the program checks for duplicate image content across the three folders using SHA-256 hashes (train.py line 109, func validate_split_independence()).

The program calculates image normalization values from training images only. It trains Logistic Regression as the baseline, then trains ClockwiseCNN using batches of 16 images. Each training epoch has the following process:

1. The CNN receives augmented training images.
2. The model produces one logit for each image.
3. BCEWithLogitsLoss compares each logit with target 1 for + or target 0 for -.
4. Backpropagation calculates gradients.
5. AdamW updates the CNN weights.
6. The program evaluates the CNN on the training and validation sets without random augmentation.
7. The program logs training loss, validation loss, training F1-score, and validation F1-score.

The maximum training length is 100 epochs. The program uses early stopping with patience 20. After each epoch, it compares the current validation loss with the best validation loss so far. When validation loss improves, the program saves the checkpoint. If validation loss fails to improve for 20 consecutive epochs, training stops early. The final model is the checkpoint with the lowest validation loss.

The test set is not used to update model weights, fit normalization values, select hyperparameters, or choose the best checkpoint. The program evaluates it only after loading the frozen best validation checkpoint.

Hyperparameters:

Hyperparameter	Value
CNN input size	128 × 128 grayscale
Batch size	16
Maximum epochs	100
Learning rate	0.001

Optimizer	AdamW
Weight decay	0.0001
Loss function	BCEWithLogitsLoss
Dropout	0.4
Early-stopping patience	20
Random seed	42

Evaluation metrics
This is a binary classification task, so the experiment uses Accuracy, Precision, Recall, F1-score, ROC-AUC, and a Confusion Matrix.

Metric	Meaning in this project
Accuracy	Proportion of all images classified correctly.
Precision	Of images predicted as +, the proportion that truly has clockwise order ABC.
Recall	Of all true + images, the proportion correctly identified as ABC.
F1-score	Harmonic means of Precision and Recall.
ROC-AUC	Measures how well the predicted probability separates + from − across thresholds.
Confusion Matrix	Shows counts of true negatives, false positives, false negatives, and true positives.

The complete dataset is close to balanced, with 104 positive and 96 negative images. However, the fixed testing split contains 19 positive and 11 negative images. Accuracy alone would not show whether the model mainly makes false-positive or false-negative errors. Precision, Recall, F1-score, ROC-AUC, and the confusion matrix provide a more complete evaluation.

6. Experimental Results and Analysis

This experiment compares two classifiers:

1. Logistic Regression, used only as a baseline reference.

2. ClockwiseCNN, used as the main model and saved for the prediction system.

Logistic Regression is trained and evaluated during every run of train.py, but it is not saved as a .pt artifact and is not used by program.py or inference.py. The saved prediction model is ClockwiseCNN.

```
(.venv) khairuonwork@Mac:~/Lab/Coding/NFU/SWE-midterm(development) $ python classification_metrics.py --checkpoint artifacts/clockwise_cnn.pt
Model                Split         accuracy  precision     recall         f1    roc_auc
-------------------------------------------------------------------------------------
Logistic Regression  train            0.986      1.000      0.970      0.985      1.000
Logistic Regression  validation       0.567      0.692      0.500      0.581      0.630
Logistic Regression  test             0.633      0.750      0.632      0.686      0.632
ClockwiseCNN         train            0.986      1.000      0.970      0.985      1.000
ClockwiseCNN         validation       1.000      1.000      1.000      1.000      1.000
ClockwiseCNN         test             0.933      1.000      0.895      0.944      0.986
```

The CNN is the selected model because it performs much better than Logistic Regression on the validation split. Its validation F1-score is 1.000, compared with 0.581 for Logistic Regression. The held-out test result confirms this difference: ClockwiseCNN achieved test F1-score 0.944, while Logistic Regression achieved 0.686.

Logistic Regression has high training performance but much lower validation and test performance. This indicates that the raw flattened pixel representation does not generalize well to unseen image layouts. The CNN preserves local and spatial image information through convolution layers, which is important for recognising the relative positions of A, B, and C.

The Logistic Regression baseline shows a clearer generalization problem: training F1-score is 0.985, but validation F1-score falls to 0.581. This large gap suggests that the baseline fits the training pixel patterns but does not generalize well.

The CNN correctly classified all 11 negative test images and produced no false positives. The only errors were two false negatives: two true positive ABC images were predicted as negative ACB. Therefore, the positive-class Recall is 0.895 rather than 1.000.

Error Case
Inference for both images such as (177_+.png and 192_+.png):

```
(.venv) khairuonwork@Mac:~/Lab/Coding/NFU/SWE-midterm(development) ✗ $ python inference.py testing/177_+.png \
  --checkpoint artifacts/clockwise_cnn.pt \
  --expected-label +

Prediction: - (clockwise ACB)
Confidence for predicted class: 0.635
Probability for +: 0.365
Expected: + (clockwise ABC)
Test result: incorrect
(.venv) khairuonwork@Mac:~/Lab/Coding/NFU/SWE-midterm(development) ✗ $ python inference.py testing/192_+.png \
  --checkpoint artifacts/clockwise_cnn.pt \
  --expected-label +

Prediction: - (clockwise ACB)
Confidence for predicted class: 0.968
Probability for +: 0.032
Expected: + (clockwise ABC)
Test result: incorrect
```

Manual inspection shows that B and C are close together in both misclassified images. This close placement likely makes the ABC and ACB order harder to distinguish after the image is resized to 128 × 128 pixels.

7. Prediction System

The project provides a complete command-line prediction system. The final prediction model is ClockwiseCNN. Logistic Regression is used only as an experimental baseline and is not used for inference.

Model saving

The program saves the CNN as a PyTorch checkpoint file. The checkpoint contains the learned state_dict and the preprocessing values needed during inference.

```
torch.save(
    {
        "state_dict": model.state_dict(),
        "mean": mean,
        "std": standard_deviation,
        "image_size": IMAGE_SIZE,
        "best_epoch": best_epoch,
        "best_validation_loss": best_validation_loss,
    },
    checkpoint_path,
)
```

Input interfaces

The project has two command-line interfaces.

1. program.py is the interactive gateway. It allows the user to choose a saved CNN checkpoint from artifacts/, review its stored test metrics, choose a numbered image from testing/, or train a new named model.
2. inference.py is the standalone prediction script. It accepts an image path and a CNN checkpoint path. This satisfies the assignment requirement for a separate inference script.

The project does not use a Gradio or Streamlit web interface. The command-line interface is the implemented prediction interface.

Inference flow

1. Input PNG image
2. Load saved CNN checkpoint
3. Convert image to grayscale
4. Resize image to saved checkpoint size, 128x128
5. Convert image to tensor
6. Normalize with mean and std from checkpoint
7. Run ClockwiseCNN
8. Apply sigmoid to obtain probability for +

9. Probability >= 0.5: Predict +
   Probability < 0.5 : Predict -
10. Print predicted class and confidence

The inference script restores the model and preprocessing settings from the checkpoint:

```
checkpoint = torch.load(checkpoint_path, map_location=device)

model = ClockwiseCNN().to(device)
model.load_state_dict(checkpoint["state_dict"])
model.eval()
```

It then applies the saved preprocessing values:

```
transforms.Compose(
    [
        transforms.Resize(
            (checkpoint["image_size"], checkpoint["image_size"])
        ),
        transforms.ToTensor(),
        transforms.Normalize(
            (checkpoint["mean"],),
            (checkpoint["std"],),
        ),
    ]
)
```

The model produces one logit. The script applies sigmoid to calculate the probability of the positive class:

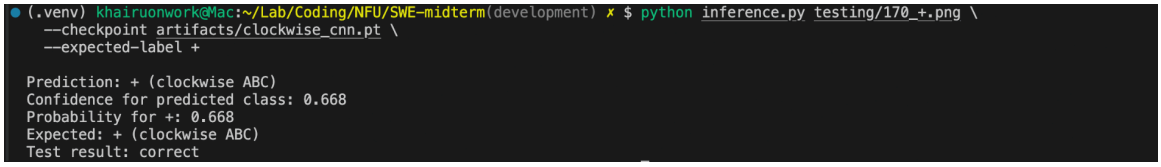
```
positive_probability = torch.sigmoid(logit).item()
```

If the probability is at least 0.5, the system predicts + (clockwise ABC). Otherwise, it predicts - (clockwise ACB). Confidence is the probability of the predicted class.

Usage example

The following command was executed using the saved CNN checkpoint:

```
python inference.py testing/170_+.png \
  --checkpoint artifacts/clockwise_cnn.pt \
  --expected-label +
```



The –expected-label option is useful only when evaluating a labelled test image. For a new unlabelled image, the user can omit it:

```
python inference.py path/to/new_figure.png \
  --checkpoint artifacts/clockwise_cnn.pt
```

Interactive usage

```
python program.py
```

The user can then select:

1. Test image
2. Train new model

For testing, the program lists available .pt CNN checkpoints in artifacts/, shows saved test metrics and lets the user choose a numbered image from testing/

Inference limitations

The models is expected to work best on images that resemble the training dataset:

- one closed curve
- readable labels A, B, and C
- a similar image scale and simple background
- label positions that are sufficiently separated
- no extra curves, labels or unrelated text

Predictions may become unstable when labels are missing, B and C are very close together, the background is noisy, several curves appear in one image, or the image layout differs substantially from the training data. The training augmentation includes only small rotation, translation, and scale changes, so extreme transformations may also reduce reliability.

Relevant implementation files are train.py (line 313), inference.py (line 24), and program.py (line 94).

8. Discussion, Limitations, and Future Improvements

Limitations

The main limitation is dataset size. The project contains 200 images, with only 140 images used for training. A small dataset can make measured performance unstable because a few difficult examples can noticeably affect validation and test metrics. The testing split contains only 30 images, so the test F1-score of 0.944 is useful evidence but does not prove broad generalization.

The images also follow a consistent visual format. They have a simple background, one coloured closed curve, and readable A, B, and C labels. The ClockwiseCNN may depend on this visual style. Its performance may decrease when an image contains a different font, noisy background, extra text, several curves, missing labels, or a different layout.

The labels were provided through filename suffixes. The project uses these labels as the supervised ground truth. No independent annotation audit or second labeller was used. If a filename label were incorrect or inconsistent, the model would learn from that incorrect target.

The CNN does not explicitly trace the curve from A to B or C. It learns statistical visual patterns from pixels. This makes the model less interpretable than a handwritten geometry algorithm. It can produce a confidence score, but it cannot fully explain which visual feature caused a prediction.

The error analysis identified two false negatives. Both were true positive ABC images where B and C were close together. Resizing to 128 × 128 pixels may reduce the visual distinction between these labels. The model therefore missed two positive examples, resulting in positive-class Recall of 0.895.

Future Improvements

More labelled images would be the most important improvement. New images should include more variation in curve shapes, label locations, fonts, line thicknesses, backgrounds, and image scales. A larger external test set should also be collected to measure generalization beyond the current fixed 30-image test split.

Label quality could be improved by reviewing the clockwise sequence for every image independently before training. A second manual check would reduce the risk of filename or annotation mistakes.

The current training augmentation uses small rotation, translation, and scaling. Future work could add controlled brightness changes, line-thickness variation, mild blur, or background noise, as long as the clockwise class remains unchanged. Horizontal and vertical flips should still be excluded because they can reverse the label order.

A stronger model could be tested after collecting more data. Possible options include a larger CNN or transfer learning with a pretrained vision model adapted for grayscale

input. These models may learn more robust spatial features, but they also increase the risk of overfitting on a small dataset.

The current dataset is close to balanced, so no class-weight adjustment was required. If future data becomes imbalanced, class-weighted loss, oversampling, or threshold tuning should be considered. The default classification threshold is currently 0.5. Threshold tuning could improve positive-class Recall if avoiding false negatives becomes more important than maintaining perfect Precision.

Feature engineering could also be explored. One approach is to detect the locations of A, B, and C first, then represent their relative positions as structured features. Another approach is to use an attention-based model that focuses on label regions. These approaches may improve interpretability, but they would require additional annotation or implementation work.

Implementation difficulties and solutions
A key challenge was distinguishing the supervised label rule from deterministic image logic. The project solved this by defining one shared mapping:

- ABC Clockwise -> + -> target 1
- ACB Clockwise -> - -> target 0

This mapping provides training targets only. The CNN still learns from image pixels rather than using a manually coded curve-tracing rule.

Another challenge was preventing data leakage. The project uses fixed training, validation, and testing folders. Normalization values are calculated from training images only, validation selects the best checkpoint, and testing is reserved for final evaluation. The program also checks SHA-256 hashes to prevent identical image content from appearing across splits.

A final challenge was making the trained model usable after training. The project solves this by saving the CNN state_dict together with preprocessing values. inference.py loads a checkpoint and predicts one image, while program.py provides a command-line menu for training and testing saved models.

9. Conclusion

This project built a supervised binary image-classification system for identifying the clockwise order of labels A, B, and C on a closed curve. The system predicts + when the clockwise order is ABC and - when the clockwise order is ACB.

The experiment compared Logistic Regression with ClockwiseCNN. Logistic Regression was used as a baseline because it classifies flattened pixel values without preserving

spatial relationships. ClockwiseCNN was selected as the final model because it learns spatial features through convolution layers.

The reproducible comparison generated by classification_metrics.py showed that ClockwiseCNN achieved the strongest held-out test performance:

Model	Test accuracy	Test F1-score	Test ROC-AUC
Logistic Regression	0.633	0.686	0.632
ClockwiseCNN	0.933	0.944	0.986

The model correctly classified 28 of 30 test images. It made two false-negative predictions, both on positive images where B and C were close together.

A usable prediction system was successfully built. The project saves the trained CNN checkpoint with its preprocessing settings, provides inference.py for standalone image prediction, and provides program.py as an interactive command-line interface for selecting a saved model, training a new model, reviewing test metrics, and testing images.

Still needed some more development and diversification of data to do before testing a wide amount of other test subjects that quite differs from the dataset.

10. References

- https://docs.pytorch.org/docs/2.15/nn.html
- https://docs.pytorch.org/docs/2.15/generated/torch.nn.BCEWithLogitsLoss.html
- https://docs.pytorch.org/docs/2.15/generated/torch.optim.AdamW.html
- https://docs.pytorch.org/docs/2.15/data.html
- https://docs.pytorch.org/vision/0.29/
- https://docs.pytorch.org/vision/stable/generated/torchvision.transforms.ToTensor.html
- https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- https://scikit-learn.org/stable/api/sklearn.metrics.html
- https://pillow.readthedocs.io/en/stable/reference/Image.html
- https://numpy.org/doc/
- https://matplotlib.org/stable/api/pyplot_summary.html

11. Appendix

The source code is available in the project GitHub repository. Rather than duplicating all source files in this report, the repository README serves as the user manual. It documents dependency installation, model training, metric comparison, and how to launch the prediction system. Additional training outputs, including metrics, learning curves, and confusion matrices, are stored in the artifacts/ directory after training.

- GitHub Link: https://github.com/khairuonwork/NFU-SE-midterm